Name: vaishnavi rajendra gayake

Subject:AIML

College name:T.b.girwalkar polytechnic college ambajogai

Branch:electronics and computer

Q2.(Strings + Loops + Functions)

```
def analyze_string(s):
    if s == "":
        print("Error: Empty string entered!")
        return
    print("Length of the string:", len(s))
    print("Reverse string:", s[::-1])
    vowels = "aeiou"
    count = 0
    for ch in s.lower():
        if ch in vowels:
            count += 1
    print("Number of vowels:", count)
    print("\nCharacter with Positive and Negative Index:")
    for i in range(len(s)):
        print(f"Positive Index: {i}, Negative Index: {i - len(s)}, Character: {s[i]}")
user_input = input("Enter a string: ")
analyze_string(user_input)
```

Q3.(Lists + Functions + Exception Handling)

```
def manage_marks():
    marks = []  # Empty list to store marks
    print("Enter marks for 5 subjects:")
    for i in range(5):
        while True:
            try:
                mark = float(input(f"Enter marks for Subject {i + 1}: "))
                marks.append(mark)
                break
            except ValueError:
                print("Invalid input! Please enter numeric marks only.")
    average = sum(marks) / len(marks)
    highest = max(marks)
    lowest = min(marks)
    marks.sort(reverse=True)
    print("\n----- Result -----")
    print("Marks List:", marks)
    print("Average Marks:", average)
    print("Highest Marks:", highest)
    print("Lowest Marks:", lowest)
manage_marks()
```

Q4.(OOP + Lists + Exception Handling)

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no
        self.marks_list = []  # Empty list to store marks
    def add_mark(self, mark):
        try:
            if mark < 0 or mark > 100:
                raise ValueError("Marks must be between 0 and 100.")
            self.marks_list.append(mark)
            print("Mark added successfully.")
        except ValueError as e:
            print("Error:", e)
    def get_average(self):
        if len(self.marks_list) == 0:
            return 0
        return sum(self.marks_list) / len(self.marks_list)
    def display_info(self):
        print("\n----- Student Information -----")
        print("Name       :", self.name)
        print("Roll No    :", self.roll_no)
        print("Marks List :", self.marks_list)
        print("Average    :", self.get_average())
try:
```

```
    name = input("Enter student name: ")
    roll_no = int(input("Enter roll number: "))
    student = Student(name, roll_no)
    n = int(input("How many marks do you want to enter? "))
    for i in range(n):
        try:
            mark = float(input(f"Enter mark {i + 1}: "))
            student.add_mark(mark)
        except ValueError:
            print("Invalid input! Please enter numeric marks.")
    student.display_info()
except ValueError:
    print("Invalid input! Roll number and number of marks must be numeric.")
```

Q5.(Dictionaries + Functions + Control Structures)

```
def student_database():
    students = {}
    while True:
        print("\n===== Student Database Menu =====")
        print("1. Add Student")
        print("2. Search Student")
        print("3. Display All Students")
        print("4. Exit")
```

```
try:
    choice = int(input("Enter your choice: "))
    if choice == 1:
        roll_no = int(input("Enter Roll Number: "))
        name = input("Enter Name: ")
        age = int(input("Enter Age: "))
        city = input("Enter City: ")
        students.update({
            roll_no: {
                "Name": name,
                "Age": age,
                "City": city
            }
        })
        print("Student added successfully.")
    elif choice == 2:
        roll_no = int(input("Enter Roll Number to search: "))
        student = students.get(roll_no)
        if student:
            print("\nStudent Found")
            print("Roll No :", roll_no)
            print("Name    :", student["Name"])
            print("Age     :", student["Age"])
            print("City    :", student["City"])
        else:
```

```
                print("Student not found.")
            elif choice == 3:
                if not students:
                    print("No student records available.")
                else:
                    print("\n----- Student Records -----")
                    for roll_no, details in students.items():
                        print(f"\nRoll No : {roll_no}")
                        print(f"Name    : {details['Name']}")
                        print(f"Age     : {details['Age']}")
                        print(f"City    : {details['City']}")
            elif choice == 4:
                print("Exiting Student Database...")
                break
            else:
                print("Invalid choice! Please select 1 to 4.")
        except ValueError:
            print("Invalid input! Please enter numeric values where required.")
student_database()
```

Q6.(Sets + Tuples + Modules)

```
import random
import math
```

```
try:
    unique_numbers = set()
    print("Enter 10 numbers:")
    for i in range(10):
        num = int(input(f"Enter number {i + 1}: "))
        unique_numbers.add(num)   # Duplicate values are automatically ignored
    numbers_tuple = tuple(unique_numbers)
    print("\nUnique Numbers (Set):", unique_numbers)
    print("Tuple:", numbers_tuple)
    if len(numbers_tuple) >= 3:
        random_numbers = random.sample(numbers_tuple, 3)
        print("3 Random Numbers:", random_numbers)
    else:
        print("Not enough unique numbers to select 3 random numbers.")
    total = sum(numbers_tuple)
    print("Sum of Tuple Elements:", total)
    if total >= 0:
        print("Square Root of Sum:", math.sqrt(total))
    else:
        print("Square root cannot be calculated for a negative sum.")
except ValueError:
    print("Invalid input! Please enter only integer numbers.")
except Exception as e:
    print("An error occurred:", e)
```

Q7.(Lambda + range() + Lists + Exception Handling)

```
square = lambda x: x ** 2
try:
    numbers = range(1, 21)
    squares = [square(num) for num in numbers]
    print("Squares of numbers from 1 to 20:")
    print(squares)
    print("\nEven Squares:")
    for value in squares:
        if value % 2 == 0:
            print(value, end=" ")
except Exception as e:
    print("An error occurred:", e)
```

Q8.(Tuples + Dictionaries + OOP)

```
class Employee:
    def __init__(self, emp_id, name, department, salary):
        self.emp_id = emp_id
        self.name = name
        self.details = (department, salary)
    def show_details(self):
```

```
        print("\nEmployee ID :", self.emp_id)
        print("Name        :", self.name)
        print("Department  :", self.details[0])
        print("Salary      :", self.details[1])
employees = {}
for i in range(3):
    print(f"\nEnter details of Employee {i + 1}")
    emp_id = int(input("Enter Employee ID: "))
    name = input("Enter Employee Name: ")
    department = input("Enter Department: ")
    salary = float(input("Enter Salary: "))
    emp = Employee(emp_id, name, department, salary)
    employees[emp_id] = emp
print("\n===== Employee Records =====")
for emp_id, emp in employees.items():
    emp.show_details()
```

Q9.(Strings + Sets + Exception Handling + Modules)

```
import math
try:
    sentence = input("Enter a sentence: ")
    if sentence.strip() == "":
        raise ValueError("Sentence cannot be empty.")
    words = sentence.split()
```

```
    unique_words = set(words)
    print("\nUnique Words (Sorted):")
    for word in sorted(unique_words):
        print(word)
    total_unique = len(unique_words)
    result = math.pow(total_unique, 2)
    print("\nTotal Unique Words:", total_unique)
    print("Unique Words Raised to Power 2:", result)
except ValueError as e:
    print("Error:", e)
except Exception as e:
    print("An unexpected error occurred:", e)
```

Q10.(Mini Project - Comprehensive)

```
import math
import random
from datetime import datetime
history = {}
def basic_arithmetic():
    try:
        num1 = float(input("Enter First Number: "))
        num2 = float(input("Enter Second Number: "))
        print("\n1. Addition")
        print("2. Subtraction")
```

```
    print("3. Multiplication")
    print("4. Division")
    choice = int(input("Enter Choice: "))
    if choice == 1:
        result = num1 + num2
        operation = f"{num1} + {num2}"
    elif choice == 2:
        result = num1 - num2
        operation = f"{num1} - {num2}"
    elif choice == 3:
        result = num1 * num2
        operation = f"{num1} * {num2}"
    elif choice == 4:
        if num2 == 0:
            raise ZeroDivisionError("Cannot divide by zero.")
        result = num1 / num2
        operation = f"{num1} / {num2}"
    else:
        print("Invalid Choice!")
        return
    print("Result =", result)
    store_result(operation, result)
except ValueError:
    print("Invalid Input! Please enter numbers only.")
except ZeroDivisionError as e:
```

```
        print("Error:", e)
def scientific_calculation():
    try:
        print("\n1. Square Root")
        print("2. Power")
        print("3. Factorial")
        choice = int(input("Enter Choice: "))
        if choice == 1:
            num = float(input("Enter Number: "))
            result = math.sqrt(num)
            operation = f"sqrt({num})"
        elif choice == 2:
            base = float(input("Enter Base: "))
            exponent = float(input("Enter Exponent: "))
            result = math.pow(base, exponent)
            operation = f"{base}^{exponent}"
        elif choice == 3:
            num = int(input("Enter Integer: "))
            result = math.factorial(num)
            operation = f"{num}!"
        else:
            print("Invalid Choice!")
            return
        print("Result =", result)
        store_result(operation, result)
```

```
    except ValueError:
        print("Invalid Input!")
    except Exception as e:
        print("Error:", e)
def generate_random():
    try:
        start = int(input("Enter Starting Number: "))
        end = int(input("Enter Ending Number: "))
        random_num = random.randint(start, end)
        print("Random Number:", random_num)
        store_result(f"Random({start},{end})", random_num)
    except ValueError:
        print("Invalid Input!")
def store_result(operation, result):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    history[timestamp] = {
        "Operation": operation,
        "Result": result
    }
    print("Result stored successfully.")
def view_history():
    if len(history) == 0:
        print("No History Available.")
    else:
        print("\n===== Calculation History =====")
```

```
        for time, details in history.items():
            print("------------------------------")
            print("Time      :", time)
            print("Operation :", details["Operation"])
            print("Result    :", details["Result"])
while True:
    print("\n========== Smart Calculator ==========")
    print("1. Basic Arithmetic")
    print("2. Scientific Calculations")
    print("3. Generate Random Number")
    print("4. View History")
    print("5. Exit")
    try:
        choice = int(input("Enter Your Choice: "))
        if choice == 1:
            basic_arithmetic()
        elif choice == 2:
            scientific_calculation()
        elif choice == 3:
            generate_random()
        elif choice == 4:
            view_history()
        elif choice == 5:
            print("Thank You! Program Closed.")
            break
```

```
    else:
        print("Invalid Choice!")
except ValueError:
    print("Please enter a valid number.")
```